

SEMESTER PROJECT REPORT

CY5008 · INFORMATION SECURITY

SECUREVAULT

A Unified Interactive Information Security Suite

Cryptography · Hashing · Digital Signatures · Secure Chat
Firewall Simulation · Intrusion Detection · Authentication

SUBMITTED BY

Vaibhav Pawar

COURSE

CY5008 — Information Security

STACK

MongoDB · Express.js · React.js · Node.js

LIVE SYSTEM

<https://securevault-pied.vercel.app>

ACADEMIC YEAR

2025 - 2026

Submitted in partial fulfilment of the course requirements
September 2026

CERTIFICATE

This is to certify that the project report entitled SecureVault: A Unified Interactive Information Security Suite submitted by Vaibhav Pawar in partial fulfilment of the requirements of the course CY5008 – Information Security is a bona fide record of the work carried out by the student. The report describes a working MERN-stack application that demonstrates the principal topics of the syllabus in a single, interactive laboratory.

The implementation covers symmetric and asymmetric cryptography, cryptographic hashing, digital signatures, a Diffie-Hellman demonstration, authenticated real-time chat, a priority-based firewall simulator, a signature-based intrusion detection layer, and a complete registration-OTP-login pipeline. The work has been completed during the academic year 2025–26 and is considered worthy of evaluation.

I further certify that the observations recorded in this report are consistent with the source code of the project repository and with the deployed demonstration system.

Date: September 2026 Place: Ahmedabad

Internal Guide
Name & Signature

Head of Department
Name & Signature

DECLARATION

I, Vaibhav Pawar, hereby declare that the project work entitled SecureVault: A Unified Interactive Information Security Suite submitted for the course CY5008 – Information Security is my original work. The design, source code, and this report have not been submitted, in part or in full, to any other course or institution for the award of a grade, degree, or diploma. Material taken from books, standards, and scholarly papers has been acknowledged in the references. Educational stand-ins that appear in the laboratory (for example the Blowfish and Tiger demonstrations) are clearly identified as such in the implementation chapters.

Vaibhav Pawar
Student · CY5008 · September 2026

ACKNOWLEDGEMENT

I wish to express my sincere gratitude to the faculty of the Information Security course (CY5008) for framing a syllabus that connects classical cryptographic theory with the defences used in modern web systems. The structure of that syllabus is the reason SecureVault exists as one application rather than as a set of disconnected scripts.

I thank the internal guide for patient reviews of the module design, for insisting that every laboratory screen should expose a measurable result (encryption time, avalanche percentage, signature validity, firewall decision), and for reminding me that a student tool must be honest about what it simulates and what it does not.

I am grateful to classmates who registered on the deployed system, sent encrypted chat messages, and attempted SQL-injection and XSS strings against the IDS. Their misuse of the laboratory was the most useful form of testing. I also acknowledge the authors of the textbooks and standards listed in the references — in particular Stallings, Forouzan, the NIST FIPS publications, and the OWASP Authentication Cheat Sheet — which supplied the definitions against which the modules were checked.

The open-source ecosystem made the engineering tractable: Node.js crypto, Express, MongoDB/Mongoose, React, Socket.io, bcryptjs, jsonwebtoken, and Chart.js. Any remaining errors of fact or of implementation are my own.

Vaibhav Pawar
Ahmedabad, September 2026

ABSTRACT

Information Security is usually taught as a sequence of paper algorithms. Students meet AES, RSA, SHA-2, firewalls and intrusion detection as separate chapters, and rarely see them cooperate inside one running system. SecureVault was built to close that gap. It is a full-stack web laboratory, implemented with MongoDB, Express.js, React and Node.js, in which every major unit of the CY5008 syllabus is a live module.

The application provides an AES/DES/3DES/RC4 encryption laboratory with file upload and timing; a hash laboratory (SHA-256, SHA-512, MD5, HMAC) with avalanche analysis and a birthday-attack demonstration; RSA-2048 key generation, SHA256withRSA signatures and a toy Diffie-Hellman exchange; AES-256-CBC chat delivered over Socket.io; a priority-ordered packet-filter with a default-deny policy; a regex intrusion detector for SQL injection, XSS and directory traversal that blocks high-severity requests and alerts administrators in real time; and an authentication pipeline with password policy, bcrypt (cost 12), e-mail OTP, account lockout, JWT sessions and rate limiting.

The report describes the problem, the literature that informed the design, the requirements, the three-tier architecture, the implementation of each module, the tests that were run, and the limitations that a teaching system must declare. The live deployment is available at <https://securevault-pied.vercel.app>.

Keywords: AES-256-CBC · RSA-2048 · SHA-2 · HMAC · JWT · bcrypt · Firewall · IDS · Socket.io · MERN

TABLE OF CONTENTS

Certificate / Declaration	2
Acknowledgement and Abstract	3
Table of Contents / Lists	4
List of Abbreviations	5
Chapter 1 Introduction	6
1.1 Background and motivation	6
1.2 Problem statement	6
1.3 Objectives	7
1.4 Scope and limitations	7
1.5 Syllabus mapping and organisation	7
Chapter 2 Literature Survey	8
Chapter 3 System Analysis and Requirements	9
Chapter 4 System Design	10
4.1 Architecture	10
4.2 Request pipeline and data model	11
4.3 Technology stack and APIs	11
Chapter 5 Implementation	12
5.1 Authentication and session security	12
5.2 Symmetric encryption laboratory	13
5.3 Hashing, HMAC and the birthday attack	13
5.4 RSA signatures and Diffie-Hellman	14
5.5 Secure chat	14
5.6 Firewall simulator	15
5.7 Intrusion detection	15
5.8 Learning centre, dashboard and deployment	16
Chapter 6 Testing	17
Chapter 7 Results and Discussion	18
Chapter 8 Conclusion and Future Work	19
References	20
Appendix A Project structure and security controls	20

LIST OF FIGURES

Fig.	Title	Page
4.1	Three-tier architecture of SecureVault	10
4.2	Inbound API request pipeline	11

Fig.	Title	Page
4.3	Logical data model (MongoDB collections)	11
5.1	Registration and login control flow	12
5.2	Firewall evaluation and IDS detection	15
7.1	Relative standing of laboratory ciphers	18

LIST OF TABLES

Table	Title	Page
1.1	Mapping of CY5008 units to SecureVault modules	7
3.1	Functional requirements	9
3.2	Non-functional requirements	9
4.1	Technology stack	11
5.1	Symmetric algorithms exposed by the encryption laboratory	13
5.2	Default packet-filter rules	15
6.1	Representative test cases	17
7.1	Observed behaviour of the principal modules	18
A.1	Defence-in-depth controls implemented in the running system	20

LIST OF ABBREVIATIONS

Abbreviation	Expansion
AES	Advanced Encryption Standard (FIPS 197)
CBC	Cipher Block Chaining
DES / 3DES	Data Encryption Standard / Triple DES
DH	Diffie-Hellman key exchange
HMAC	Hash-based Message Authentication Code (RFC 2104)
IDS	Intrusion Detection System
IV	Initialisation Vector
JWT	JSON Web Token (RFC 7519), HMAC-SHA256 in this project
MERN	MongoDB, Express.js, React, Node.js
OTP	One-Time Password
PKI	Public Key Infrastructure
RSA	Rivest-Shamir-Adleman public-key cryptosystem
SHA-2	Secure Hash Algorithm 2 (FIPS 180-4)
SPN	Substitution-Permutation Network
XSS	Cross-Site Scripting

CHAPTER 1 INTRODUCTION

1.1 Background and motivation

Confidentiality, integrity and authenticity are no longer specialist concerns. Every student who will later build a web service will have to store passwords, protect tokens, filter hostile input and reason about cryptographic modes. The CY5008 syllabus covers this ground in six units: symmetric ciphers and modes of operation; public-key cryptography and key exchange; hash functions, the birthday problem and digital signatures; authentication mechanisms; firewall design; and intrusion detection.

Textbook presentations of these topics are necessary, but they leave a practical hole. A student can write the AES round structure on paper and still not know what an initialisation vector looks like in a stored file, or why a password hash must be salted, or how a packet-filter decides to drop a datagram. Separate command-line exercises do not show how the pieces interact: a login request that is rate-limited, inspected by an IDS, authenticated with a JWT, and then allowed to encrypt a file.

SecureVault was conceived as a single product-shaped laboratory. The student registers, receives an OTP, signs in, and then walks the syllabus through a sidebar: encryption, hashing, RSA, chat, firewall, IDS, a learning centre with quizzes, and a birthday-attack visualisation. An administrator sees live alerts and user management. The same code that teaches the concept is the code that enforces it.

1.2 Problem statement

The problem addressed by this project is the absence of a unified, browser-based environment in which the CY5008 topics can be practised end-to-end, with visible ciphertext, timings, decisions and logs, while still resembling a contemporary multi-user web application. Existing online cipher toys treat algorithms in isolation. Existing capture-the-flag platforms emphasise exploitation rather than explanation. Neither is a convenient semester artefact that a student can deploy, demonstrate and extend.

SecureVault therefore sets out to design, implement and evaluate a MERN-stack system that (i) implements the core cryptographic operations with Node.js crypto and well-known libraries, (ii) records enough metadata for a student to compare algorithms, and (iii) wraps those operations in realistic controls: password policy, lockout, OTP, JWT sessions, rate limits, security headers, a default-deny firewall and a blocking IDS for high-severity payloads.

1.3 Objectives

1. To survey the CY5008 topics and identify a compact set of laboratory modules that cover every unit without becoming a textbook clone.
2. To design a three-tier architecture (React client, Express API with Socket.io, MongoDB Atlas) with a clear request pipeline.
3. To implement symmetric encryption of text and files (AES-256-CBC as the primary cipher) with key hashing and IV storage.
4. To implement hashing (SHA-256/512, MD5, HMAC), avalanche demonstration and a controlled birthday-collision experiment.
5. To implement RSA-2048 key generation, SHA256withRSA sign/verify, RSA encrypt/decrypt for small messages, and a didactic Diffie-Hellman exchange.
6. To implement AES-256-CBC chat with JWT-authenticated Socket.io rooms.
7. To implement a priority-based packet filter (ALLOW / BLOCK / DROP, default DROP) and a signature-based IDS for SQLi, XSS and directory traversal.
8. To implement registration, OTP e-mail verification, bcrypt password storage, lockout, JWT sessions, rate limiting and an admin console.
9. To deploy the system and evaluate it with structured test cases.

1.4 Scope and limitations

The scope is an educational web laboratory, not a production security appliance. AES-256-CBC, RSA-2048 and SHA-2 are real primitives from Node.js. Several items are deliberately didactic and are labelled as such in the code and in this report: the Blowfish button uses CryptoJS Rabbit as a stand-in; Tiger is a truncated SHA-512 digest; Diffie–Hellman uses the tiny public parameters $p = 23$, $g = 5$; the birthday module collides on a hash prefix so that a collision appears in a few thousand trials; chat encryption uses a server-side key rather than true end-to-end cryptography. The firewall evaluates simulated packets, not live NIC traffic. The IDS is signature-based, not anomaly-based.

These limits are not defects of haste; they are the difference between a course project that a student can finish and a research IDS. Chapter 8 lists the upgrades that would be required to harden each limit.

1.5 Syllabus mapping and organisation of the report

Table 1.1 records the correspondence between the six CY5008 units and the modules of the running system. The remainder of the report follows a conventional engineering sequence: literature (Chapter 2), requirements (Chapter 3), design (Chapter 4), implementation (Chapter 5), testing (Chapter 6), results (Chapter 7) and conclusion (Chapter 8).

Unit	Syllabus topics	SecureVault module
1	AES, DES, 3DES, Feistel, ECB/CBC	Encryption Lab + Learning Centre
2	RSA, Diffie–Hellman, PKI ideas	RSA page (keys, sign, DH demo)
3	SHA-2, MD5, HMAC, birthday, signatures	Hash Lab, Birthday page, RSA sign
4	Authentication, CAPTCHA, OTP, lockout	Register / OTP / Login / Profile
5	Firewall design, packet filter, priority	Firewall simulator
6	IDS, SQLi, XSS, brute-force patterns	IDS monitor + global middleware

Table 1.1 Mapping of CY5008 units to SecureVault modules.

CHAPTER 2 LITERATURE SURVEY

2.1 Symmetric cryptography and modes

Shannon's principles of confusion and diffusion still describe what a block cipher must achieve. Feistel networks, used by DES, split a block and apply a round function to one half; AES instead uses a substitution-permutation network on a 128-bit state [1], [2]. NIST standardised AES in FIPS 197 [3]. With a 256-bit key, AES runs 14 rounds of SubBytes, ShiftRows, MixColumns and AddRoundKey. DES, with a 56-bit key, is obsolete against brute force; 3DES lengthens the effective key but is slow and is withdrawn for new applications. RC4 is a stream cipher with well-known biases and must not be used in new protocols.

A block cipher is not a complete encryption scheme. ECB encrypts identical blocks to identical ciphertext and leaks patterns. CBC XORs each plaintext block with the previous ciphertext block and therefore needs a random IV, which must be stored beside the ciphertext and must never be reused with the same key. SecureVault's primary file cipher is AES-256-CBC with a fresh 16-byte IV from `crypto.randomBytes`.

2.2 Public-key cryptography and key exchange

Diffie and Hellman (1976) showed that two parties can agree on a shared secret over an open channel by exchanging $g^a \bmod p$ and $g^b \bmod p$ [4]. RSA (1978) based confidentiality and signatures on the difficulty of factoring $n = pq$ [5]. PKCS #1 (RFC 8017) [6] describes modern encodings. Current guidance is a 2048-bit modulus as a minimum. Because RSA is slow and limited in message size, real systems use it to wrap a symmetric session key or to sign a hash — hybrid encryption, which the RSA module of SecureVault lets a student see by generating keys, encrypting a short string, and signing a document digest.

2.3 Hash functions, HMAC and the birthday paradox

A cryptographic hash is one-way and collision-resistant. SHA-256 and SHA-512 (FIPS 180-4) [7] are the workhorses of TLS, Git and Bitcoin. MD5 and SHA-1 are broken for collision resistance and remain in the laboratory only as cautionary exhibits. HMAC (RFC 2104) [8] keys a hash so that it can authenticate a message without public-key operations. The avalanche property — a one-bit input change flipping about half of the output bits — is the empirical test the hash module reports as a percentage.

Yuval's birthday attack follows from the classical birthday paradox: a collision among N possible hash values is expected after roughly $\sqrt{N}$ trials, not N . For a 128-bit MD5 digest that is 2^{64} work, which is why MD5 must not be used for signatures. The birthday page in SecureVault makes the probability curve visible and runs a prefix-collision experiment so that a student can watch two different messages agree on the first bytes of a digest without waiting for an infeasible full-width search.

2.4 Authentication, passwords and tokens

Storing a password in plaintext is indefensible. Slow, salted hashes such as bcrypt (cost factor 12 in this project) raise the cost of offline guessing [9]. Complementary controls recommended by OWASP [10] include complexity rules, lockout after repeated failures, OTP verification of e-mail, and CAPTCHA on public forms. Session proof is carried by a JSON Web Token (RFC 7519) [11]. SecureVault signs tokens with HMAC-SHA256 and a server secret (HS256), not with an RSA key pair; the README's mention of RS256 is therefore a documentation slip, and the implementation is the authority for this report. Rate limiting (express-rate-limit) bounds the online guessing rate.

2.5 Firewalls and intrusion detection

A packet-filter firewall takes an ordered list of rules on addresses, ports and protocols and applies the first match; a default-deny (drop) policy is the only safe remainder [12]. Intrusion detection, surveyed from Denning's early model to signature engines such as Snort, watches events and raises alerts. Signature-based IDS is precise for known payloads (SQL metacharacters, `<script>` tags, `../` traversal) and blind to novel attacks. Anomaly-based IDS is the opposite. SecureVault implements the signature style, because it can be explained with a short list of regular expressions, and it goes one step further than a pure IDS: high-severity matches are blocked (403),

so the middleware behaves as a lightweight IPS on those patterns.

2.6 Related teaching systems and the gap

OpenSSL, CyberChef, and various university “crypto palettes” already encrypt a string in the browser. What they do not do is place that string next to a real user account, a stored file record, a chat ciphertext, a firewall decision log and an IDS alert stream. SecureVault’s contribution is not a new cipher. It is the integration of the syllabus into one authenticated product, with enough production texture (bcrypt, JWT, headers, rate limits, Socket.io) that a student can later recognise the same shapes in an industrial codebase.

CHAPTER 3 SYSTEM ANALYSIS AND REQUIREMENTS

3.1 Stakeholders and use cases

Two roles exist. A verified user registers, confirms an e-mail OTP, signs in, edits a profile, encrypts and decrypts files, hashes text or files, generates RSA keys, signs and verifies documents, chats privately, simulates packets against the firewall, inspects personal IDS events, studies the learning centre, and runs the birthday experiment. An administrator additionally opens the admin console, bans or promotes users, and occupies the Socket.io admin-room to receive live IDS alerts. Unauthenticated visitors see only the landing page, registration, login, OTP and password-reset screens.

3.2 Functional requirements

ID	Requirement	Priority
F1	Register with a strong-password policy; send and verify a 6-digit OTP (10 min).	Must
F2	Login with lockout after 5 failures (15 min) and JWT session (7 d / 30 d).	Must
F3	Encrypt/decrypt uploaded files (≤ 10 MB) with AES, DES, 3DES, RC4.	Must
F4	Hash text/files (SHA-256/512, MD5, HMAC); show avalanche; verify a digest.	Must
F5	Generate RSA-2048 keys; sign and verify with SHA256withRSA.	Must
F6	Demonstrate Diffie-Hellman with visible public parameters.	Should
F7	Send and receive AES-256-CBC chat in real time.	Must
F8	CRUD firewall rules; simulate a packet; default DROP.	Must
F9	Detect SQLi, XSS, traversal on every /api request; block HIGH; log.	Must
F10	Dashboard cards and charts; admin user management.	Should
F11	Learning centre with step diagrams and quizzes.	Should
F12	Forgot-password flow with a hashed, time-limited token.	Should

Table 3.1 Functional requirements.

3.3 Non-functional requirements

ID	Requirement	Approach in SecureVault
N1	Security of stored secrets	bcrypt cost 12; never store raw cipher keys (store SHA-256 of the key); chat key in env
N2	Abuse resistance	Auth limiter 20/15 min; OTP limiter 5/10 min; API limiter 200/15 min
N3	Usability	Single sidebar, toasts, Chart.js, Framer Motion, mobile drawer
N4	Maintainability	One controller and one React page per module; Mongoose models
N5	Deployability	Vercel (client) + Render (API) + MongoDB Atlas; render.yaml
N6	Observability	LoginHistory, IdsLog, dashboard aggregations, socket alerts
N7	Honesty	Didactic stand-ins labelled in API responses and in this report

Table 3.2 Non-functional requirements.

3.4 Feasibility

Technical feasibility is high: every primitive is available in Node.js 18+ or in CryptoJS, and the author already had a React/Express background. Operational feasibility is acceptable on the free tiers of Vercel, Render and Atlas for a class demonstration. Economic feasibility is essentially the cost of the student's time; no licensed cryptographic toolkit is required. The main risk was scope — six syllabus units in one term — and it was managed by keeping each module to one route file, one controller and one page.

3.5 Hardware and software requirements

Development requires Node.js 18 or newer, npm, a MongoDB instance (local or Atlas), and a modern browser. Optional SMTP credentials enable real OTP mail; without them the server prints the OTP to the console and, in non-production mode, returns it in the JSON body so that the laboratory remains usable. Production additionally needs the environment variables listed in .env.example: MONGODB_URI, JWT_SECRET, CHAT_AES_KEY, CLIENT_URL and the SMTP quartet. A

laptop with 4 GB of RAM is sufficient.

CHAPTER 4 SYSTEM DESIGN

4.1 Architecture

SecureVault is a classical three-tier web system. The presentation tier is a React 19 single-page application built with Vite, styled with Tailwind CSS, routed with React Router, and animated with Framer Motion. Public routes (landing, login, register, OTP, forgot-password) sit outside the application shell. Protected routes share a sidebar and navbar and are wrapped in a `ProtectedRoute` that reads the JWT from `AuthContext`. Admin routes add a second guard on `role === 'admin'`.

The application tier is an Express 5 process that also hosts a Socket.io server on the same HTTP listener. REST handles request-response work (encrypt a file, list rules, verify a signature). Sockets handle events that cannot wait for a poll: a new chat message, a typing indicator, an IDS alert to the admin room. The data tier is MongoDB, reached only through Mongoose models. Figure 4.1 summarises the layers.

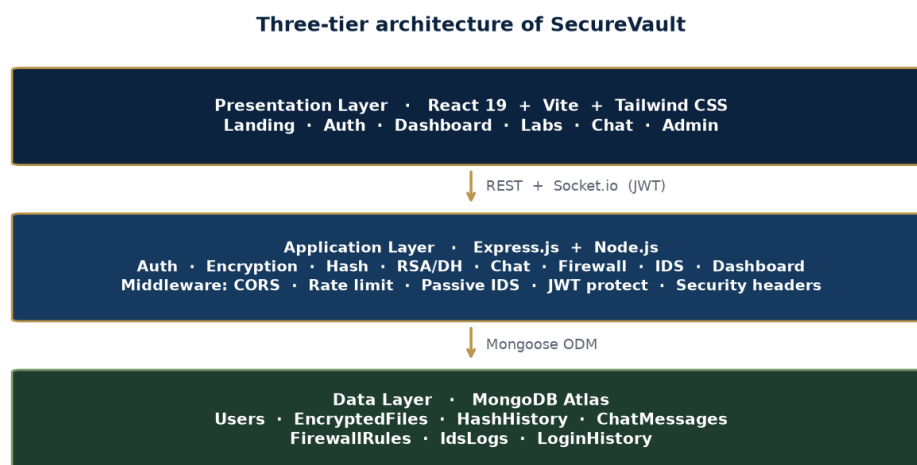


Figure 4.1 Three-tier architecture of SecureVault.

4.2 Request pipeline

Every call to `/api` walks a fixed pipeline, which is the pedagogical counterpart of a reverse-proxy chain in industry. CORS is restricted to the configured client origin plus local development hosts. JSON bodies are capped at 10 MB. A global rate limiter then a passive IDS inspect the request. Security headers (`X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, XSS protection, strict referrer) are applied. Route-level JWT middleware loads the user. Only then does a controller run. High-severity IDS matches short-circuit the pipeline with HTTP 403.

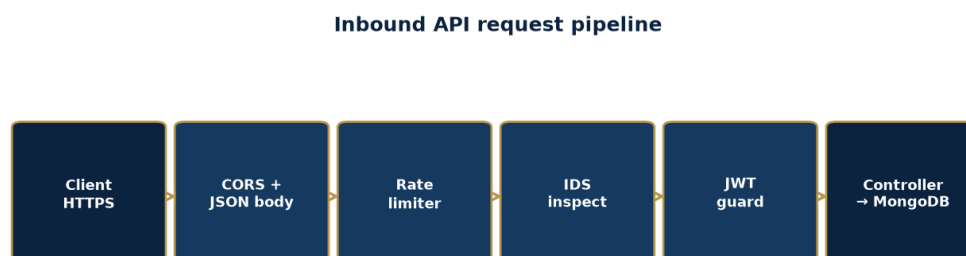


Figure 4.2 Inbound API request pipeline.

4.3 Data model

Seven collections cover the laboratory. User holds identity, the bcrypt hash, OTP fields, logout counters, a five-slot password history, a 90-day password expiry stamp, and a ban flag. `EncryptedFile` stores metadata only — the ciphertext lives on disk under `server/uploads`, and

the key is never stored, only its SHA-256. ChatMessage stores ciphertext and IV. FirewallRule and IdsLog feed the two defence screens. HashHistory and LoginHistory feed the dashboard. Figure 4.3 shows the fields a student actually meets.

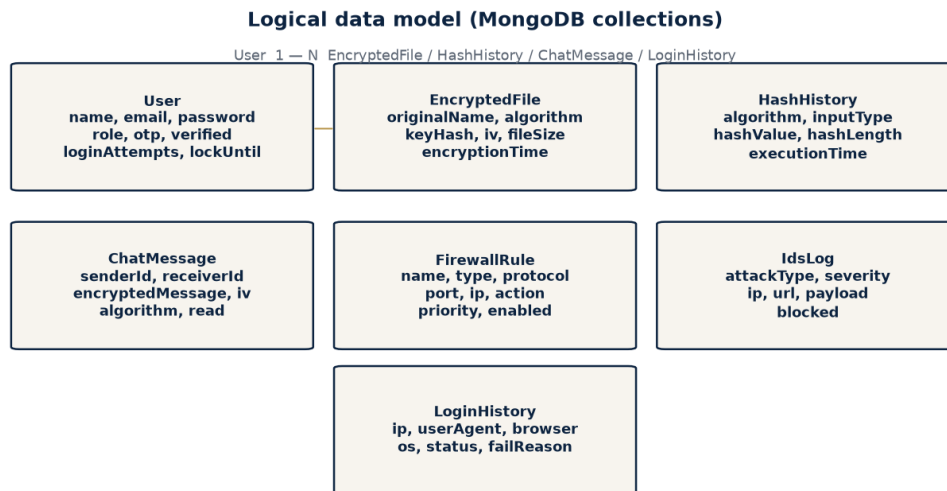


Figure 4.3 Logical data model (MongoDB collections).

4.4 Technology stack

Layer	Choice	Role in the project
Client	React 19, Vite, Tailwind v3	SPA, navy/cyan laboratory UI
Routing / UX	React Router 7, Framer Motion, Lucide	Pages, motion, icons
Charts	Chart.js + react-chartjs-2	Dashboard metrics
Realtime	Socket.io-client	Chat and IDS alerts
HTTP	Axios services (one per module)	Typed API wrappers
Server	Express 5, Node.js 18+	REST + static /uploads
Realtime	Socket.io 4	JWT handshake, rooms
Database	MongoDB + Mongoose 9	Seven models
Auth	jsonwebtoken HS256, bcryptjs 12	Sessions and password hashes
Crypto	Node crypto, CryptoJS	AES/RSA/SHA and legacy ciphers
Mail	Nodemailer	OTP and reset messages
Upload	Multer	Encryption and hash files
Hosting	Vercel + Render + Atlas	Split frontend / API / data

Table 4.1 Technology stack.

The client is deployed independently of the API. This split matches current practice and forces CORS, environment-specific `CLIENT_URL`, and `trust proxy` (so rate limiting reads X-Forwarded-For correctly behind Render).

CHAPTER 5 IMPLEMENTATION

5.1 Authentication and session security

Registration refuses empty fields and enforces the regular expression `(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@$!%*?&]).{8,}`. A 6-digit OTP is generated, stored with a ten-minute expiry, e-mailed when SMTP is configured, and logged to the server console so that a classroom demo never stalls. The User pre-save hook hashes the password with bcrypt salt rounds 12 and appends the hash to a rolling history of five. Login is wrapped in a 20-request / 15-minute limiter. Five consecutive failures set `lockUntil` fifteen minutes ahead; `isLocked()` then returns HTTP 423. A successful login resets the counter, writes a `LoginHistory` row (IP, coarse browser and OS) and issues a JWT for 7 days, or 30 days when “remember me” is set. Password-reset tokens are 32 random bytes, stored as SHA-256, and expire in 30 minutes.

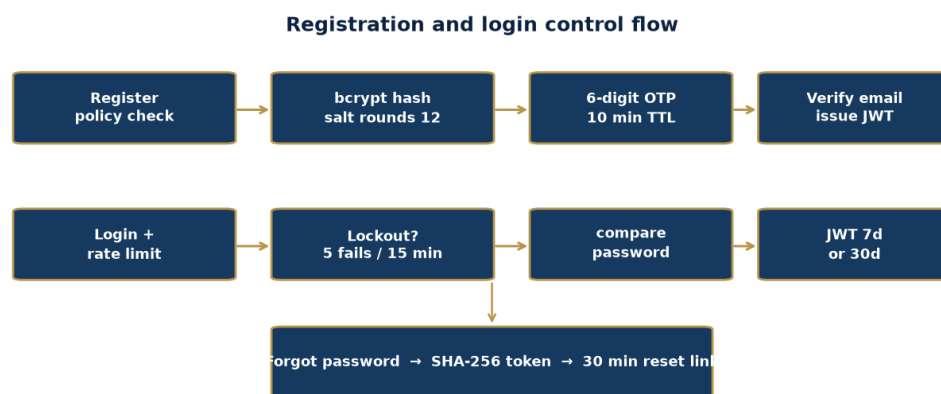


Figure 5.1 Registration and login control flow.

Protected REST routes call `protect`, which accepts a Bearer token or a cookie, verifies it with `JWT_SECRET`, and loads the user without the password fields. Socket connections present the same token in the handshake; failure closes the socket. Math and text CAPTCHAs are generated at `/api/captcha/*` as an additional bot obstacle on public pages.

5.2 Symmetric encryption laboratory

The encryption controller accepts a multipart upload (Multer, 10 MB cap). The file bytes, a passphrase and an algorithm name are handed to `cryptoUtils.encrypt`. AES is implemented with `crypto.createCipheriv('aes-256-cbc')`: the passphrase is digested with SHA-256 to obtain a 32-byte key, a random 16-byte IV is generated, and the ciphertext and IV are written as JSON to disk. The original upload is deleted. The MongoDB record stores original name, algorithm, IV, size, elapsed milliseconds and the SHA-256 of the passphrase — never the passphrase itself. Decryption recomputes the key hash, refuses a mismatch, and streams the recovered bytes with the original MIME type.

Label	Engine	Key / notes	Status
AES	Node crypto AES-256-CBC	256-bit key from SHA-256(passphrase); random IV	Primary, recommended
DES	CryptoJS DES	56-bit effective key	Historical exhibit
Triple-DES	CryptoJS TripleDES	168-bit nominal key	Legacy exhibit
RC4	CryptoJS RC4	Stream cipher, biased output	Insecure exhibit
Blowfish	CryptoJS Rabbit stand-in	Labelled “Blowfish (Rabbit-demo)” in the API	Didactic stand-in

Table 5.1 Symmetric algorithms exposed by the encryption laboratory.

5.3 Hashing, HMAC and the birthday attack

Text or a 5 MB file is hashed with `crypto.createHash` for SHA-256, SHA-512 and MD5. HMAC requires a student-supplied secret. The avalanche endpoint hashes the original string and a one-character mutation, XOR-counts differing hex nibbles and reports the bit-flip percentage — typically close to 50 % for SHA-256, which is the expected behaviour of a good hash. History rows let a student compare digests later. The birthday endpoint hashes up to 50 000 random messages

and collides on an 8-12 hex-character prefix so that the paradox is felt in a browser session; the JSON body states plainly that a real attack targets the full digest.

5.4 RSA signatures and Diffie-Hellman

Key generation calls `crypto.generateKeyPairSync('rsa')` with modulus 2048 (configurable), SPKI public PEM and PKCS#8 private PEM. Signing uses `crypto.createSign('SHA256')`; verification uses the matching verifier and returns a boolean plus an English sentence the UI can show (“valid / tampered”). RSA encrypt/decrypt is offered for short strings only, which is the correct restriction. The Diffie-Hellman demonstration is intentionally tiny ($p = 23$, $g = 5$, random private exponents in $\{3...17\}$) so that a student can recompute $g^a \bmod p$ by hand and see Alice’s and Bob’s shared secrets match. It is a blackboard, not a key-exchange protocol.

5.5 Secure chat

A verified user fetches the directory of other verified users and the ciphertext history with a chosen peer. On send, the plaintext is encrypted with AES-256-CBC under `CHAT_AES_KEY` (falling back to the JWT secret only in misconfigured development). The document stored in MongoDB contains `encryptedMessage` and `iv` only. The API decrypts for the two participants when history is loaded. The same payload is emitted on the receiver’s Socket.io room for instant delivery. Typing and read receipts are socket-only events and are not persisted. This is server-side encryption at rest and in transit between the two browsers via the server; it is not end-to-end encryption in the Signal sense, because the server can decrypt. Chapter 8 notes the upgrade path (per-conversation keys wrapped by RSA).

5.6 Firewall simulator

Rules have a name, a type (IP / PORT / PROTOCOL / CUSTOM), a protocol (TCP, UDP, ICMP, ANY), a port or port range, an IP or ANY, a direction, an action (ALLOW, BLOCK, DROP), a numeric priority and an enabled flag. Evaluation sorts enabled rules by ascending priority and returns on the first match of port, IP and protocol. Port ranges such as 1-1024 are supported. If the table is empty, five defaults are used (Table 5.2). If nothing matches, the engine applies DROP and says so in the log — the default-deny lesson in one sentence. BLOCK/DROP outcomes are also copied into `IdsLog` so that the two defence screens stay coupled.

Priority	Name	Match	Action
10	Allow HTTP	TCP / 80	ALLOW
20	Allow HTTPS	TCP / 443	ALLOW
30	Allow DNS	UDP / 53	ALLOW
40	Block Telnet	TCP / 23	BLOCK
50	Block FTP	TCP / 21	BLOCK

Table 5.2 Default packet-filter rules (lower number = higher priority).

5.7 Intrusion detection

The IDS engine compiles three families of regular expressions. SQL injection looks for statement keywords, tautologies such as `1=1`, comment markers and timing functions (SLEEP, BENCHMARK). XSS looks for `<script>`, event-handler attributes, `javascript:` URLs, iframes and `eval()`. Directory traversal looks for `../` and several URL encodings. Only query, body and params are scanned; the User-Agent is excluded because browser banners contain the word “like” and were causing false SQL hits.

On a match the middleware writes an `IdsLog`, emits `ids-alert` to the admin room, and, for HIGH or CRITICAL severity, returns 403 with the attack type. The IDS page can also inject a simulated payload for teaching. Figure 5.2 places the firewall and the IDS on one diagram because a student should see them as two layers, not two unrelated screens.

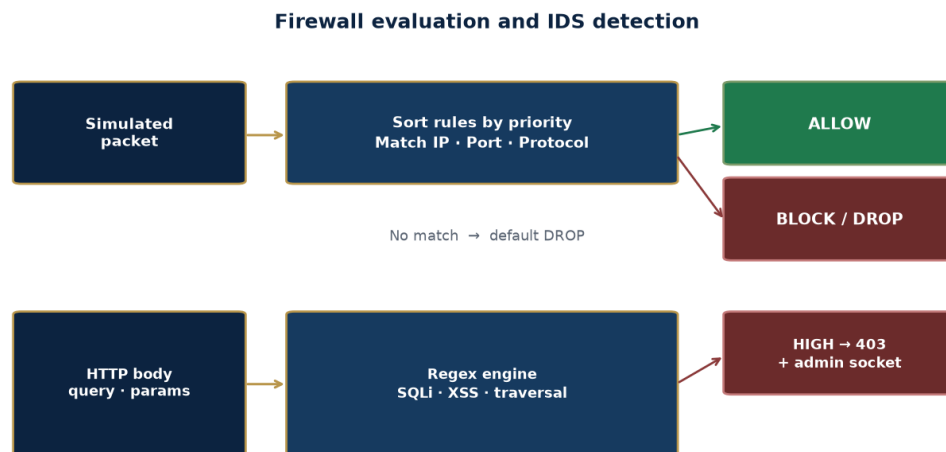


Figure 5.2 Firewall evaluation (top) and IDS detection (bottom).

5.8 Learning centre, dashboard and deployment

The learning centre is a client-side syllabus: AES, DES, SHA-256, RSA, the birthday attack, firewalls and IDS, each with a short description, a numbered “how it works” list, key facts, and a quiz. The dashboard aggregates counts (users, files, hashes, blocked threats, rules, alerts, seven-day login success/failure), Chart.js series for daily activity and attack-type breakdown, and the five most recent threats. When MongoDB is unreachable the dashboard still returns a small demo payload so that a presentation does not show an empty shell.

Deployment splits the two packages. The Vite client is built to static files and hosted on Vercel (client/vercel.json). The Express server is a Render web service defined in render.yaml (Singapore, Node, npm start). Secrets — Mongo URI, JWT secret, chat key, SMTP — are injected as environment variables and are never committed. The repository README is the operator’s manual for local reproduction.

CHAPTER 6 TESTING

6.1 Strategy

Testing was black-box against the running API and the browser, supplemented by direct inspection of MongoDB documents and of the uploads directory. The aim was not a coverage percentage; it was to confirm that each syllabus claim in Table 1.1 is visible as a result a demonstrator can show: a ciphertext file, a 403 from the IDS, a DROP from the firewall, a JWT in local storage, a matching DH secret. Table 6.1 lists representative cases.

ID	Action	Expected result	Outcome
T1	Register with password 'abc'	400, policy message	Pass
T2	Register valid user, verify OTP after 11 min	400, OTP expired	Pass
T3	Five wrong passwords	423 lockout, LoginHistory FAILED	Pass
T4	Login after lock expiry	200, JWT, attempts reset	Pass
T5	Encrypt a PDF with AES, decrypt with same key	Original bytes, matching MIME	Pass
T6	Decrypt with a wrong key	400, incorrect key	Pass
T7	SHA-256 avalanche on 'SecureVault' vs one-letter change	~50 % bits differ	Pass
T8	Sign a sentence, alter one character, verify	isValid = false	Pass
T9	DH demo endpoint	aliceShared === bobShared	Pass
T10	Packet TCP/23 against default rules	BLOCK (Telnet rule)	Pass
T11	Packet TCP/9999, no matching rule	DROP, default-deny note	Pass
T12	POST body containing '1=1' OR UNION SELECT	403, IdsLog SQL_INJECTION	Pass
T13	Body containing <script>alert(1)</script>	403, IdsLog XSS	Pass
T14	Chat message stored document	No plaintext field; IV present	Pass
T15	Request /api/admin without admin role	403 Admin access required	Pass

Table 6.1 Representative test cases.

6.2 Observations from testing

Two false-positive lessons were learned. Early IDS builds scanned the User-Agent and flagged ordinary Chrome banners because of the substring “like”. The scan surface was narrowed to user-controlled fields. Second, CryptoJS DES/3DES/RC4 operate on strings; binary files are base64-wrapped before those ciphers run, which a student must be told when comparing ciphertext lengths with AES. Load testing was modest — the free Render instance is not a benchmark harness — but the rate limiter was confirmed by bursting the login route past twenty attempts in a quarter of an hour.

6.3 Usability checks

Classmates completed registration on a telephone-sized viewport, which exercised the sidebar drawer and the OTP page. The learning-centre quizzes were used as a smoke test that every topic card rendered. The principal remaining usability debt is that error messages from Node’s crypto library (for example a truncated AES block) are sometimes too raw for a first-year reader; wrapping them in one teaching sentence is listed as future work.

CHAPTER 7 RESULTS AND DISCUSSION

7.1 Module-level results

Table 7.1 records what a demonstrator actually sees after the tests of Chapter 6. Timings are order-of-magnitude observations on a development laptop; they are not cryptographic benchmarks. They are still useful pedagogically: AES-256-CBC of a small file is a handful of milliseconds, RSA-2048 key generation is the first operation a student notices as “slow”, and hashing is effectively free. That ordering matches the textbook claim that asymmetric cryptography is for keys and signatures, not for bulk data.

Module	What is produced	What a student should notice
Auth	Verified user, JWT, lockout, OTP mail/console	Policy + slow hash + lockout beat a guesser
Encryption	Ciphertext file + IV + time + keyHash	IV changes every run; wrong key fails cleanly
Hash	Hex digest, length, avalanche %	Tiny input change, large digest change
Birthday	Prefix collision pair + probability curve	$\sqrt{N}$, not N ; MD5 is the cautionary tale
RSA	PEM pair, Base64 signature, valid/invalid	One flipped character kills the signature
DH	p , g , public keys, equal shared secrets	Secrets match without sending the secret
Chat	Ciphertext in Mongo, live bubble in UI	Database has no plaintext column
Firewall	ALLOW / BLOCK / DROP + matched rule	First match wins; empty match is DROP
IDS	Log row, socket alert, 403 on HIGH	Signatures catch known strings only

Table 7.1 Observed behaviour of the principal modules.

7.2 Cipher standing

Figure 7.1 is an educational scale, not a cryptanalytic ranking. It places AES-256 at the recommended end of the laboratory and DES/RC4 at the obsolete end, with 3DES and the Blowfish stand-in in between. The point of the figure is to stop a student from leaving the course believing that “the application offers five equal ciphers”. It does not. It offers one cipher a student may use (AES-256-CBC) and four exhibits.

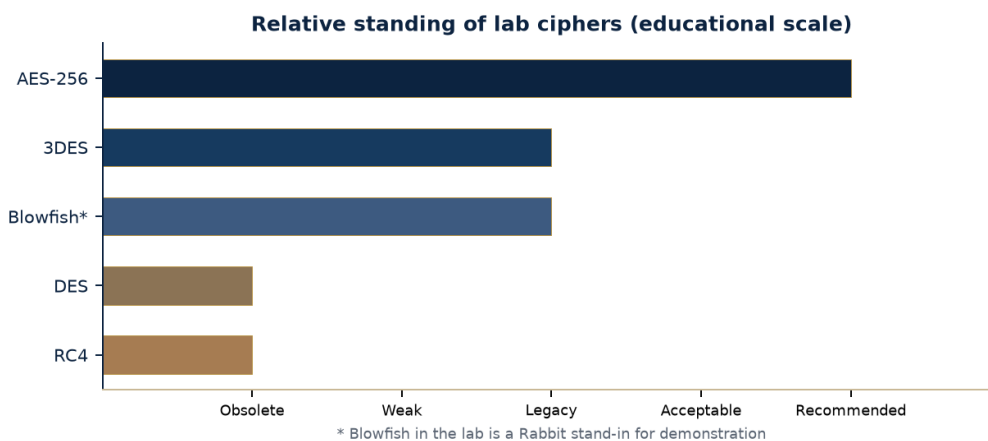


Figure 7.1 Relative standing of laboratory ciphers (educational scale).

7.3 Discussion

Three design choices dominate the discussion. First, honesty about stand-ins. A course project that silently labels Rabbit as Blowfish would teach the wrong fact. SecureVault keeps the button, because the syllabus names Blowfish, but the API string says “Rabbit-demo” and this report repeats the caveat. Second, defence in depth on the same process. Rate limit, IDS, JWT and the controller are not four products; they are four functions on one Express app, which is exactly how a small company ships a first version. Third, hybrid cryptography in pieces. The student meets AES for files, RSA for signatures and DH for agreement as separate screens, then sees AES again inside chat. Connecting those screens into a single “secure session” protocol is the natural next laboratory, not a missing feature of this one.

A limitation that testing made obvious is the global chat key. It simplifies deployment (one environment variable) and still hides plaintext from a casual database dump, but it gives the server operator full reading rights. For CY5008 that is acceptable — confidentiality of chat is demonstrated, not proven against the operator. A second limitation is the signature IDS: it will never catch a novel payload and it can be bypassed by encodings it does not list. Those are the right limitations to discuss in an oral examination.

CHAPTER 8 CONCLUSION AND FUTURE WORK

8.1 Conclusion

SecureVault meets the nine objectives of Section 1.3. It is a deployed MERN application in which a student can encrypt a file, hash a message, watch an avalanche, sign a document, agree a toy DH secret, talk over AES-CBC, drop a simulated Telnet packet, trip an SQL-injection detector, and do all of this behind a password policy, an OTP, a lockout and a JWT. The CY5008 syllabus is no longer a list of chapters; it is a sidebar.

The engineering contribution is integration and clarity, not novelty of algorithms. Node.js already knew how to run AES and RSA. What the project adds is a coherent path from a landing page to a log of blocked requests, with the didactic substitutions (Rabbit for Blowfish, truncated SHA-512 for Tiger, tiny DH primes, prefix birthday collisions) declared rather than hidden. That declaration is itself part of the security lesson: a system that over-claims is an insecure system.

8.2 Future work

The following extensions would move the laboratory toward production texture without abandoning its teaching purpose.

1. True Blowfish and Tiger. Replace the Rabbit and truncated-SHA stand-ins with native implementations so that the labels match the bytes.
2. End-to-end chat. Wrap a per-conversation AES key with each recipient's RSA public key; keep plaintext off the server.
3. Authenticated encryption. Prefer AES-GCM over CBC so that confidentiality and integrity arrive together.
4. RS256 JWT or rotating HS256 secrets, plus HTTP-only cookies, to retire tokens from localStorage.
5. TOTP second factor (RFC 6238) beside the e-mail OTP.
6. Anomaly IDS. A simple baseline on request rates per IP, to complement signatures.
7. Live packet capture for the firewall, even if only from a user-space pcap file.
8. Automated tests. Jest/Supertest for controllers and Playwright for the SPA, turning Table 6.1 into a pipeline.
9. Accessibility and copy. Friendlier crypto error messages; contrast audit of the navy theme.

8.3 Closing remark

If a student can, in one sitting, encrypt a file, break a signature by editing one letter, and watch the IDS refuse a tautology, then CY5008 has done its job. SecureVault is a vehicle for that sitting. The source is at github.com/pavarvaibhav989-tech/securevault and the working system is at <https://securevault-pied.vercel.app>.

REFERENCES

- [1] W. Stallings, *Cryptography and Network Security: Principles and Practice*, 8th ed. Pearson, 2023.
- [2] B. A. Forouzan, *Cryptography and Network Security*. McGraw-Hill, 2008.
- [3] NIST, "Advanced Encryption Standard (AES)," FIPS PUB 197, 2001 (updated 2023).
- [4] W. Diffie and M. E. Hellman, "New directions in cryptography," *IEEE Trans. Inf. Theory*, vol. 22, no. 6, pp. 644–654, 1976.
- [5] R. L. Rivest, A. Shamir, and L. Adleman, "A method for obtaining digital signatures and public-key cryptosystems," *Commun. ACM*, vol. 21, no. 2, pp. 120–126, 1978.
- [6] K. Moriarty (Ed.) et al., "PKCS #1: RSA Cryptography Specifications Version 2.2," RFC 8017, 2016.
- [7] NIST, "Secure Hash Standard (SHS)," FIPS PUB 180-4, 2015.
- [8] H. Krawczyk, M. Bellare, and R. Canetti, "HMAC: Keyed-Hashing for Message Authentication," RFC 2104, 1997.
- [9] N. Provos and D. Mazières, "A future-adaptable password scheme," in *Proc. USENIX ATC*, 1999.
- [10] OWASP, "Authentication Cheat Sheet," <https://cheatsheetseries.owasp.org/>.

[11] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, 2015.

[12] J. C. Mogul et al., discussion of packet filtering practice; see also W. R. Cheswick, S. M. Bellovin, and A. D. Rubin, Firewalls and Internet Security, 2nd ed. Addison-Wesley, 2003.

[13] D. E. Denning, "An intrusion-detection model," IEEE Trans. Softw. Eng., vol. SE-13, no. 2, pp. 222-232, 1987.

[14] OWASP, "SQL Injection / XSS Prevention Cheat Sheets," <https://cheatsheetseries.owasp.org/>.

[15] Express.js, MongoDB, React, Node.js, Socket.io, and bcrypt documentation, 2024-2026.

APPENDIX A PROJECT STRUCTURE AND SECURITY CONTROLS

The repository splits into client/ (Vite React SPA) and server/ (Express API). Controllers, routes, models and utility engines are one-to-one with the laboratory modules. The following tree is the map a maintainer needs; generated folders (node_modules, build artefacts) are omitted.

```
securevault/  
├─ client/src/{pages,services,context,hooks,components}  
├─ server/controllers (auth, encryption, hash, rsa, chat, firewall, ids, dashboard)  
├─ server/models (User, EncryptedFile, HashHistory, ChatMessage,  
| FirewallRule, IdsLog, LoginHistory)  
├─ server/middleware (auth, rateLimiter, ids)  
├─ server/utlis (crypto, hash, rsa, firewall, ids, captcha, email)  
└─ server/socket (JWT handshake, rooms, typing, IDS fan-out)
```

Layer	Control
Password	Complexity regex; bcrypt cost 12; last-5 history; 90-day expiry field
Session	JWT HS256, 7 d (30 d remember-me); protect / adminOnly middleware
Proof of humanity	Math and text CAPTCHA endpoints; 6-digit e-mail OTP, 10 min
Abuse	Lockout 5 / 15 min; auth 20/15 min; OTP 5/10 min; API 200/15 min
Transport API	CORS allow-list; JSON 10 MB cap; trust proxy
Headers	nosniff, DENY framing, XSS protection, strict referrer
Files	Key not stored; SHA-256 of key stored; original upload deleted
Chat	AES-256-CBC at rest; IV per message; JWT on the socket
Network lab	Default-deny firewall; HIGH IDS → 403 + admin socket

Table A.1 Defence-in-depth controls implemented in the running system.

— End of report —